

Universal Optimality of Dijkstra via Beyond-Worst-Case Heaps

论文评述

摘要

Haeupler、Hladik、Rozhon、Tarjan 和 Tetek 的 *Universal Optimality of Dijkstra via Beyond-Worst-Case Heaps* 研究 Dijkstra 算法在 beyond-worst-case 视角下的最优性。本文评述以 2025 年 5 月 7 日发布的 arXiv v4 扩展版为准；该版本在 FOCS 2024 会议论文基础上重写并改进了若干证明。文章讨论的核心任务是 **distance order problem**：给定源点 s ，输出顶点按照从 s 出发的真实距离非递减排列的顺序。这个任务弱于完整 SSSP，因为它不要求输出所有距离数值；但它保留了 Dijkstra 算法最关键的扫描顺序，也适合用比较次数和图拓扑本身的顺序自由度来衡量。

文章证明，在合适的 **priority queue** 支持下，普通 Dijkstra 在运行时间上已经达到 **universal optimality**；进一步处理图中的 **bottleneck** 后，Dijkstra 的两个扩展版本还能在比较次数上达到相应下界。除了下界和算法，作者还构造了支持 **decrease-key** 的 **working-set heap**，使时间上界不依赖额外的数据结构假设。文章关注的并非继续改进一般图上的 **worst-case** 上界，而是解释固定拓扑如何影响 Dijkstra 的图相关复杂度。

1. 论文解决的问题
2. 为什么这个问题重要
3. 相关研究
4. 主要技术路线
5. 技术直觉
6. 关键技术细节
- 6.1 下界

$\Omega(m)$ 下界来自输入访问。若算法没有检查某条可能影响距离顺序的边，就可以通过调整这条边的长度制造一个与算法输出冲突的实例。

$\Omega(\log D)$ 是决策树下界。固定图可能产生 D 个 **distance order**，对每个 **order** 都能构造边权使其成为目标输出。正确算法必须区分这些叶子，因此比较深度至少为 $\log D$ 。

$\Omega(F - n + 1)$ 的作用是补上 $\log D$ 之外的前向弧信息。这里不能简单理解成每条额外前向弧都必然对应一次比较；更准确地说， $F - n + 1$ 是论文在比较下界和 Dijkstra 比较次数分析中共同使用的图参数，用来度量基本可达性之外的前向弧自由度。

这个下界的核心是变量数和约束数的比较。选取一个有 F 条 **forward arcs** 的 **distance order** $L = [v_1, \dots, v_n]$ ，并给前向弧 $v_i v_j$ 赋长度 $j - i$ 。若某个算法只做至多 $F - n$ 次比较，那么这些比较最多固定 $F - n$ 个线性约束；再加入 $n - 1$ 个相邻顶点距离差约束，约束总数仍少于 F 个前向弧长度变量。于是还存在一个非零扰动方向，可以保持算法见到的比较结果不变，同时逐步改变某些前向弧长度。沿这个方向移动到某条非相邻前向弧长度变为零时，算法的比较记录没有变化，但原顺序 L 已不再是正确的 **true distance order**；算法若仍输出 L ，就会出错。

6.2 Dijkstra 与 working-set bound

Dijkstra 的扫描顺序确实是 **true distance order**。证明可以用通常的反证法：设当前被 **delete-min** 取出的顶点是 u ，但仍有未扫描顶点 v 满足 $d(v) < d(u)$ 。取一条从 s 到 v 的最短路，并令 x 是这条路上第一个尚未扫描的顶点， y 是它的前驱。由于 y 已经扫描过，松弛边 (y, x) 后有 $\hat{d}(x) \leq d(x) \leq d(v) < d(u) \leq \hat{d}(u)$ ，这与 u 是队列中估计距离最小的顶点矛盾。

运行中，堆里保存 **labeled** 但尚未 **scanned** 的顶点。在固定的扫描顺序下，只有从较早扫描顶点指向较晚扫描顶点的弧，才可能参与对后续标签的有效更新。论文把每个非源点所需的一条基本前驱弧与其余前向弧分开，后者进入 $F - n + 1$ 项。这里不能理解成每条额外前向弧都机械对应一次比较；更准确地说， $F - n + 1$ 是下界和 Dijkstra 比较次数分析中共同使用的图参数。

delete-min 的分析更细。设 Dijkstra 的插入序列为 $v_1, \dots, v_n$ 。对 v_i ，令 $a_i = i$ ，令 b_i 表示它被删除时已经插入的最后一个顶点下标，那么 v_i 的堆生命周期就是区间 $[a_i, b_i]$ ，并且 $W(v_i) = b_i - a_i + 1$ 。**decrease-key** 只改变 **key**，不改变这个区间的左右端点。

这些区间构成一个 **interval DAG** I ：若区间 i 在区间 j 开始前已经结束，即 $b_i < a_j$ ，就加弧 $i \rightarrow j$ 。论文引用的区间 DAG 引理给出 $\sum_i \log(b_i - a_i + 1) = O(\log \#Top(I))$ ，其中 $\#Top(I)$ 是 I 的拓扑序数量。剩下要做的是把 $\#Top(I)$ 接到图本身的 D 。

令 T 为 Dijkstra 第一次标记每个顶点时形成的树。如果树边是 $v_i \rightarrow v_j$ ，那么 v_i 已经被删除后， v_j 才因扫描这条边而第一次插入堆，所以 $b_i < a_j$ ，这条树边也是 **interval DAG** 中的边。于是， I 的任意拓扑序都尊重 T 。根据论文第 3 节的判定，一个有根树的任意拓扑序都是原图的 **distance order**，因此 $\#Top(I) \leq D$ 。合在一起就得到 $\sum_v \log W(v) = O(\log D)$ 。这一步把 **working-set heap** 的局部删除代价转化成了固定图可产生的顺序数量。

6.3 Lookahead

Lookahead 版本先按无权层数处理图。这里 $\ell(v)$ 是从 s 到 v 的路径中最少顶点数；若某一层只有一个顶点，它就是 **bottleneck**，并且会支配所有更高层顶点。**Marked bottleneck** 指下一层至少有两个顶点的 **bottleneck**；**unmarked bottleneck** 则是最高层顶点，或下一层仍然只有一个顶点的 **bottleneck**。

算法只把非 **bottleneck** 放进堆 H 。未处理的 **bottleneck** 按层数存入数组 B ， B 一直取到第一个 **marked bottleneck** 为止。扫描普通顶点时，边松弛仍按 Dijkstra 的方式更新标签；若被更新的是非 **bottleneck**，就插入 H 或执行 **decrease-key**，若是 **bottleneck**，则只更新它在 B 中的距离记录。

主循环比较两类候选： H 中最小非 **bottleneck** 的 **tentative distance**，以及 B 中当前 **bottleneck** 段能够推出的距离。若堆顶更小，就删除并扫描这个非 **bottleneck**。否则，算法沿 B 从低层到高层扫描；对连续 **unmarked bottleneck**，论文用 Lemma 8.3 说明下一个 **bottleneck** 的真实距离可由前一个 **bottleneck** 和相应出边得到。若整段 B 的最大距离仍不超过堆顶，就把整段加入输出序列；否则只加入 B 中距离不超过堆顶的最长前缀。

正确性可以看成是一个安全性引理：当算法把某个 **bottleneck** b 加入输出序列时，所有尚未输出的非 **bottleneck** 顶点 x 都满足 $d(b) \leq d(x)$ 。原因是低层 **bottleneck** 支配更高层区域，堆顶又代表所有非 **bottleneck** 候选中的最小 **tentative distance**；算法只有在显式比较后才推进 **bottleneck** 段。因此提前输出这些 **bottleneck** 不会打乱 **true distance order**。

复杂度还要解释为什么堆里剩下的非 **bottleneck** 不会太多。设图中有 b 个 **bottleneck**，**level** 总数为 r 。每个不含 **bottleneck** 的 **level** 至少有两个顶点，所以 $r \leq (n+b)/2$ 。取一棵 BFS 树后，在同一 **level** 内任意排列顶点，可以得到至少 $\prod_i |V_i|! \geq 2^{n-r}$ 个树的拓扑序，也就是原图的 **distance orders**。于是 $\log D \geq n - r \geq (n - b)/2$ ，非 **bottleneck** 顶点数 $n - b$ 本身就是 $O(\log D)$ 。这一步支付了普通堆中非 **bottleneck** 的基本 **delete-min** 成本。

还有两项记账容易被略过。对 **delete-min**，论文把 **bottleneck** 也虚拟地看成“立即插入、立即删除”的元素，再复用 **working-set bound** 的区间论证，得到总成本仍为 $O(\log D)$ 。对数组 B 中的指数搜索和二分搜索，给每个非 **bottleneck** 顶点 v 记下它跨过的 **bottleneck** 集合 $B(v)$ ；不同选择会产生至少 $\prod_v (|B(v)| + 1)$ 个 **distance orders**，因此搜索成本 $\sum_v \log(|B(v)| + 1)$ 也能由 $O(\log D)$ 控制。

6.4 Recursive Dijkstra

Recursive Dijkstra 把 **bottleneck** 当作递归边界。初始化时，所有 **bottleneck** 已按层数放入列表 L 。调用 **Dijkstra**(u) 时，算法为这个源点建立一个局部堆，先扫描 u ，然后反复删除局部堆的最小顶点；如果删出的顶点是 **bottleneck**，就以它为新源

点递归调用 **Dijkstra**(v)，否则扫描它并把它插入最终列表 L 。

这里的递归不是在整张图上反复重跑 **Dijkstra**。算法维护全局的 **scanned/****labeled** 状态，每个顶点只扫描一次，相关出边也只由负责它的那次扫描处理。多个局部堆表示的是被 **bottleneck** 切开的若干前沿；较高 **level** 的 **bottleneck** 所对应的递归运行在扫描边时，可能改进一个已经存放在较低 **level** 递归堆中的顶点。此时算法直接在该顶点当前所属的堆中执行 **decrease-key**，而不是把它移动到当前堆。这样递归深度不会把 m 乘起来。

维护 L 要用 **finger search tree**。正确性上，非 **bottleneck** 顶点 v 被扫描时，它的父亲 $p(v)$ 已在 L 中，且 $d(p(v)) \leq d(v)$ 。算法从 $p(v)$ 附近向后搜索第一个距离至少为 $d(v)$ 的顶点，把 v 插在它前面；前后距离关系保证了 L 仍按真实距离非递减排列。

复杂度上，令 $v_1, \dots, v_n$ 为顶点的扫描顺序。若 $p(v_i) = v_j$ ，则为 v_i 定义区间 $[j+1, i]$ 。**Finger search** 的秩距离至多由这个区间长度控制，所以插入代价是 $O(1 + \log(i - j))$ 。再对这些区间应用 **Lemma 7.1**，就能把所有插入成本控制在 $O(\log D)$ 。

多个局部堆的 **delete-min** 成本也要合并计算。论文主要考虑 **marked bottleneck** 对应的递归调用；对每个这样的 **bottleneck** v ，取搜索树中由该调用负责的局部子树 $T(v)$ 。该局部运行的 **working-set** 成本可由 $O(\log \# \text{Top}(T(v)))$ 控制。全局搜索树的拓扑序数量至少包含这些局部子树拓扑序数量的乘积，取对数后，各局部成本可以相加，最终仍被 $O(\log D)$ 吸收。

6.5 Working-set heap

Outer heap 由一串 **inner heaps** 组成：

$$H_1, H_2, \dots, H_k。$$

编号小的 **heap** 放较新元素，编号大的 **heap** 放较旧元素。每个 **inner heap** 可用 **Fibonacci heap**、**hollow heap** 或 **rank-pairing heap** 这类 **fast heap** 实现。插入新元素时先建立 H_0 ，再检查相邻层是否可以 **meld**。大小阈值采用双指数形式，大致保持 $|H_i| \leq 2^{2^i}$ ，从而保证层数只有 $O(\log \log n)$ 。

Outer heap 对外不提供 **meld**；下表中的 **meld** 只指 **insert** 过程中对两个 **inner fast heaps** 的内部合并。

各操作的运转方式可以概括如下：

操作	做法	摊还代价
<code>insert(x)</code>	新建最年轻层，必要时按大小阈值逐层 <code>meld</code> 和重新编号	$O(1)$
<code>decrease-key(x)</code>	用 <code>union-find</code> 定位 x 所在 <code>inner heap</code> ，在内部减键并更新外层最小值信息	$O(1)$
<code>find-min()</code>	通过 <code>suffix-minimum bit vector</code> 找到含全局最小 <code>key</code> 的 <code>inner heap</code>	$O(1)$
<code>delete-min(x)</code>	在对应 <code>inner heap</code> 中删除 x ，再更新该层及更年轻层的 <code>suffix minimum</code> 标记	$O(\log W(x))$
内部 <code>meld</code>	仅在 <code>insert</code> 中合并满足大小条件的相邻 <code>inner heaps</code> ，并同步执行一次集合 <code>unite</code>	计入 <code>insert</code> 的摊还代价

`delete-min` 不会把 `inner heap` 分裂，只是在内部删除元素并修正外层索引；真正改变分层结构的主要是后续插入触发的内部 `meld` 和 `reindex`。Inner heaps 合并时，对应的元素集合执行一次 `unite`；`decrease-key(x)` 通过 `find(x)` 找到 x 所属的 `inner heap`。论文不在元素删除时同步拆分集合。对 Dijkstra 产生的操作序列，查找过程中产生的额外父指针变化可摊入该元素最终的 $O(\log W(x))$ 删除代价；在允许元素长期不被删除的一般情形下，作者再使用 `fixed-tree set union` 获得严格的 $O(1)$ 摊还界。`Suffix-minimum bit vector` 的常数时间依赖 `word-RAM` 操作，以及 `inner heap` 数量只有 $O(\log \log n)$ 这一事实；若 n 事先未知，论文还用重建技巧维持空间和字长假设。

7. 技术贡献与局限

8. 总结

参考文献

- [1] Bernhard Haeupler, Richard Hladík, Václav Rozhoň, Robert E. Tarjan, and Jakub Tětek. 2025. *Universal Optimality of Dijkstra via Beyond-Worst-Case Heaps*. arXiv:2311.11793v4. Substantially rewritten and improved version of the FOCS 2024 paper.
- [2] Edsger W. Dijkstra. 1959. *A note on two problems in connexion with graphs*. Numerische Mathematik.
- [3] Michael L. Fredman and Robert E. Tarjan. 1987. *Fibonacci heaps and their uses in improved network optimization algorithms*. Journal of the ACM.
- [4] Michael L. Fredman, Robert Sedgwick, Daniel D. Sleator, and Robert E.

- Tarjan. 1986. *The Pairing Heap: A New Form of Self-Adjusting Heap*. Algorithmica.
- [5] Bernhard Haeupler, Siddhartha Sen, and Robert E. Tarjan. 2011. *Rank-Pairing Heaps*. SIAM Journal on Computing.
 - [6] Thomas D. Hansen, Haim Kaplan, Robert E. Tarjan, and Uri Zwick. 2017. *Hollow Heaps*. ACM Transactions on Algorithms.
 - [7] Amr Elmasry. 2006. *A Priority Queue with the Working-Set Property*. International Journal of Foundations of Computer Science.
 - [8] Amr Elmasry, Arash Farzan, and John Iacono. 2012. *A priority queue with the time-finger property*. Journal of Discrete Algorithms.
 - [9] Scott Huddleston and Kurt Mehlhorn. 1982. *A New Data Structure for Representing Sorted Lists*. Acta Informatica 17, 157-184. doi:10.1007/BF00288968.
 - [10] Tim Roughgarden. 2020. *Beyond the Worst-Case Analysis of Algorithms*. Cambridge University Press.
 - [11] Bernhard Haeupler, David Wajc, and Goran Zuzic. 2021. *Universally-Optimal Distributed Algorithms for Known Topologies*. STOC 2021.
 - [12] Bernhard Haeupler, Richard Hladik, John Iacono, Vaclav Rozhon, Robert E. Tarjan, and Jakub Tetek. 2025. *Fast and Simple Sorting Using Partial Information*. SODA 2025.
 - [13] Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu, and Longhui Yin. 2025. *Breaking the Sorting Barrier for Directed Single-Source Shortest Paths*. arXiv:2504.17033.
 - [14] Ivor van der Hoog, Eva Rotenberg, and Daniel Rutschmann. 2025. *Simpler Universally Optimal Dijkstra*. ESA 2025, LIPIcs 351, 71:1-71:9. doi:10.4230/LIPIcs.ESA.2025.71. arXiv:2504.17327.
 - [15] Bernhard Haeupler, Richard Hladik, Vaclav Rozhon, Robert E. Tarjan, and Jakub Tetek. 2025. *Bidirectional Dijkstra’s Algorithm is Instance-Optimal*. 8th SIAM Symposium on Simplicity in Algorithms (SOSA 2025), 202-215. doi:10.1137/1.9781611978315.15.
 - [16] Sandeep Sen. 2025. *Towards universally optimal sorting algorithms*. arXiv:2506.08261.

- [17] Benjamin Aram Berendsohn. 2026. *Universally Optimal Decremental Tree Minima*. arXiv:2602.15977.
- [18] Mikkel Thorup. 2004. *Integer Priority Queues with Decrease Key in Constant Time and the Single Source Shortest Paths Problem*. Journal of Computer and System Sciences 69(3), 330-353. doi:10.1016/j.jcss.2004.04.003.